Resource-Constrained Neural Monte Carlo Tree Search for Quantum Boolean Oracle Synthesis

Zixiang Zhou

July 9, 2026

Abstract

Boolean oracle synthesis embeds classical predicates into quantum algorithms, but direct algebraic-normal-form constructions can be costly in non-Clifford gates, CNOTs, logical depth, and temporary workspace. We formulate bit-flip Boolean oracle synthesis as a resource-constrained search problem over ANF/FPRM term sets and introduce Resource-NMCTS, a logical-layer synthesis framework combining neural action priors, Monte Carlo tree search, Boolean-ring linear-factor screening, depth-frontier policies, Pareto archives, and baseline-preserving guards. The method does not use the SSHR parallelotope candidate space; SSHR is used only as a CNOT-oriented small-function baseline. On 177 traditional functions with $n \leq 6$, Pareto-Resource-NMCTS reduces mean T-count by 72.25% and mean weighted score by 67.80% relative to direct ANF synthesis. Against ESOP beam, weighted ESOP-MILP, and SSHR-H, its score win/loss/tie counts are 174/0/3, 167/3/7, and 173/4/0, respectively. External probes strengthen this comparison. Against a ROS-style LUT proxy over 309 functions, Pareto-Resource-NMCTS obtains 309/0/0 score wins with a mean 84.27% reduction. Against an official-header mockturtle KLUT-to-XAG probe, it obtains 166/11/0 wins on the traditional set and 64/0/0 wins on an $n = 14$ high-dimensional set. A CirKit 3 AIG/multiplicative-complexity probe returns verified rows for 177 traditional and 64 high-dimensional functions; Pareto-Resource-NMCTS wins all score comparisons, with mean reductions of 62.34% and 94.46%, but loses most depth comparisons. A legacy RevKit CLI probe embeds each function as the exact reversible oracle permutation $(x, y) \mapsto (x, y \oplus f(x))$ and runs TBS/DBS/RMS synthesis flows. All 531 direct flow rows return; against the per-function RevKit best-score portfolio, Pareto-Resource-NMCTS has 173/0/4 score wins and a mean score reduction of 67.28%, but uses more peak ancilla on 169/177 functions. High-dimensional correctness is supported by 5640/5640 item-level symbolic runs, 1512/1512 width-probe symbolic runs, 531/531 phase selected rows, and 400/400 complete truth table-bridge method rows for $n = 21$–25. The contribution is therefore a verified logical-layer search method with strong T-count and weighted-score advantages, not a hardware-mapped or depth-only synthesis claim.

1 Introduction

Given a Boolean function $f : \{0, 1\}^n \to \{0, 1\}$, a bit-flip oracle implements

$$|x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle. \tag{1}$$

Such oracles appear in Grover search, constraint solving, and quantum cryptanalysis. Their cost is not determined only by the number of Boolean operations. In a fault-tolerant or resource-limited setting, the chosen Boolean representation, compute/uncompute order, temporary workspace, and phase realization all affect T-count, CNOT count, logical depth, gate count, and peak ancilla.

Existing synthesis routes provide strong local solutions. ESOP, XAG, LUT and BDD representations reduce different Boolean resources; ROS makes LUT-based oracle synthesis resource-aware; RevKit, mockturtle, CirKit, and ABC provide mature logic and reversible-synthesis toolchains; and SSHR uses parallelotope structures in the Boolean hypercube to target small-scale CNOT-oriented synthesis [6, 5, 4, 13, 1, 15, 11, 10, 9]. These methods motivate the baselines used in this paper, but they also expose a gap: a fixed representation or a single resource objective may miss useful tradeoffs when T-count, CNOTs, depth, gates, and ancilla are optimized together.

We address this gap by treating Boolean oracle synthesis as an explicit search problem. The state is an ANF/FPRM term set; actions are algebraic rewrites that can be expanded back to GF(2) semantics; neural policies and MCTS allocate the search budget; and guard rules ensure that learned candidates do not replace a stronger deterministic baseline. The resulting circuit is checked at the plan level, at the emitted X/CNOT/MCT symbolic level, and, for bridge instances, by full truth table oracle simulation.

The paper makes four contributions. First, it defines Resource-NMCTS as a resource-constrained ANF/FPRM search pipeline rather than as an SSHR extension. Second, it combines Boolean-ring linear-factor screening, neural action priors, MCTS, depth-frontier policies, Pareto archives, and baseline-preserving guards into one logical-layer synthesis workflow. Third, it evaluates the method against ESOP, SSHR, ABC/BDD, ROS-style LUT, mockturtle, CirKit, RevKit CLI, and RevKit phase/Rz baselines on matched functions. Fourth, it separates algorithmic gains from engineering guards through ablations and high-dimensional verification bridges.

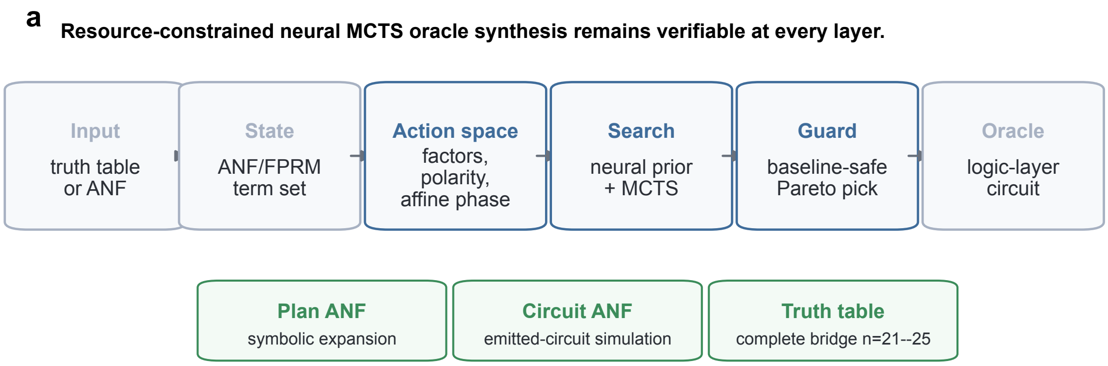

Figure 1: **Verified search pipeline.** Resource-NMCTS operates on complete truth table inputs or ANF term sets, searches algebraic actions under neural and tree-search control, and verifies candidate circuits at plan, emitted symbolic-circuit, and complete-truth table levels.

2 Related Work

2.1 Boolean representations for oracle synthesis

Low-T oracle synthesis is closely tied to the Boolean intermediate representation. BDD-based reversible synthesis uses symbolic decision diagrams to handle large functions [13]. ROS frames oracle synthesis as resource-constrained LUT mapping with downstream reversible implementation costs [6]. XAG-based compilation separates XOR structure from AND nodes, making multiplicative complexity a proxy for non-Clifford cost [5, 4]. Back-end-aware oracle synthesis further connects logic synthesis to downstream resource metrics, but that line includes mapping concerns that are outside the scope of this logical-layer study [14].

2.2 Reversible and logic-synthesis toolchains

ABC supplies mature AIG, XAG/GIA, LUT, and ESOP optimization flows for classical logic [1]. mockturtle and CirKit provide reusable logic-network optimization libraries and shells [10, 9]. RevKit provides reversible-logic and quantum-compilation functionality, including the Python `oracle_synth` API and legacy command-line reversible synthesis flows [11]. In this work these toolchains are used as external baselines or probes. They are not called inside Resource-NMCTS.

2.3 Learning-guided synthesis

Learning-guided synthesis has been explored for both classical and quantum circuits. ShortCircuit uses AlphaZero-style search for classical Boolean circuit design [12]. Gumbel AlphaZero has been used for Clifford+T unitary synthesis [7]; reinforcement learning has been used for non-Clifford-count optimization and ZX-calculus rewriting [2, 8]; and MonteQ uses MCTS for quantum circuit synthesis [3]. The present work is different in scope: the learned component ranks verifiable Boolean-algebraic actions rather than generating arbitrary gate sequences.

3 Problem and Resource Model

We represent a Boolean function by its square-free ANF

$$f(x) = \bigoplus_{m \in \mathcal{M}} \prod_{i \in m} x_i, \tag{2}$$

where each monomial m is stored as a bit mask. A direct construction applies one controlled action per monomial. The synthesis problem is to replace this direct construction with reusable compute/action/uncompute subplans that implement Eq. (1) while lowering a multi-resource objective.

All resource numbers in this paper are logical-layer estimates over X, CNOT and multi-controlled Toffoli gates. Candidate selection uses

$$\text{score} = T + 0.04\,\text{CNOT} + 0.015\,\text{depth} + 0.01\,\text{gates} + 2\,\text{ancilla}. \tag{3}$$

The score is a ranking objective, not a physical runtime or error model. We therefore report T-count, CNOT count, depth, gate count, and peak ancilla separately whenever a tradeoff is relevant.

4 Method

4.1 Search state and actions

The Resource-NMCTS state is the current set of unsynthesized ANF terms. An action chooses a reusable algebraic structure, synthesizes that structure into a temporary line, uses it to control a quotient subproblem, and then uncomputes it. Candidate actions include common monomial factors, FPRM polarity rewrites, CNOT-computed linear parity factors, Boolean-ring linear factors, and bounded affine changes of variables. Each action is scored by its immediate resource estimate and by the size and structure of the induced subproblems.

4.2 Boolean-ring linear factors

A key extension is allowing the linear factor and the quotient to share variables under Boolean-ring semantics. For example,

$$(x_i \oplus x_j)x_i m = x_i m \oplus x_i x_j m, \tag{4}$$

because $x_i^2 = x_i$ over the Boolean ring. Circuit-wise, the parity $x_i \oplus x_j$ is computed by CNOTs into a temporary line, used as a control for the quotient plan, and uncomputed. This expands the algebraic action space without changing the GF(2) verifier.

4.3 Neural prior, MCTS, frontier policy, and guard

The neural action prior ranks candidate actions using term-count, degree, coverage, variable-support, and immediate-resource features. MCTS then explores combinations of actions with deterministic rollout estimates. A separate frontier policy selects among depth-2, depth-3, and depth-4 Boolean screening frontiers; a cost-aware version trades a small score increase for lower planning time and reduced ancillary lifetime area. Finally, a guard compares learned or expensive candidates to deterministic baselines and returns the best valid candidate under Eq. (3). This makes learning a search-control component, not the source of semantic correctness.

4.4 Pareto archive

The Pareto variant evaluates candidates under several internal objectives, including active-score, T-sparse, CNOT/depth-heavy, gate-sensitive, and ancilla-tight rankings. It removes candidates dominated simultaneously in T-count, CNOT count, depth, gate count, and peak ancilla, then selects from the remaining archive using the active score. The archive is used only after each candidate has passed the same plan and emitted-circuit checks as the base method.

4.5 Phase/Rz branch

RevKit `oracle_synth` returns phase/Rz netlists rather than direct bit-flip Clifford+T circuits. To avoid treating that boundary only as a cost sensitivity, we also implement a phase-parity emitter: each ANF monomial is expanded into merged parity-phase gadgets and verified up to global phase. Fixed-polarity FPRM search and bounded affine preconditioning are then added as phase-search actions. This branch is reported as a logical phase/Rz proxy; it does not output approximate rotation sequences or mapped hardware circuits.

5 Experimental Design

Experiments answer four questions. First, does Resource-NMCTS reduce T-count and weighted score on small functions where full truth table checking is practical? Second, do the conclusions survive comparisons with ESOP, SSHR, ABC/BDD, ROS-style LUT, mockturtle, CirKit, RevKit CLI, and RevKit phase/Rz baselines? Third, which components–affine search, MCTS, neural priors, guards, frontier policies, and Pareto archives–account for the gains? Fourth, how far can correctness evidence be pushed beyond small truth tables?

The traditional slice contains 177 functions with $n \leq 6$. High-dimensional logic-network probes include exported $n = 14$, $n = 15$, $n = 16$, and $n = 18$ random-ANF sets for ABC/BDD, and $n = 14$ sets for mockturtle and CirKit. Scale and bridge experiments cover item-level $n = 20$–40 term sets and complete truth table bridge functions at $n = 21$–25. Paired comparisons include only matching functions or items with successful semantic checks and no timeout.

6 Results

6.1 Traditional functions

Figure 2 and Table 1 summarize the traditional $n \leq 6$ slice. Pareto-Resource-NMCTS has the lowest mean T-count and weighted score among the shown internal baselines, reducing T-count by 72.25% and score by 67.80% relative to direct ANF synthesis. SSHR-H retains the lowest mean CNOT count, so the result is not a CNOT-only dominance claim.

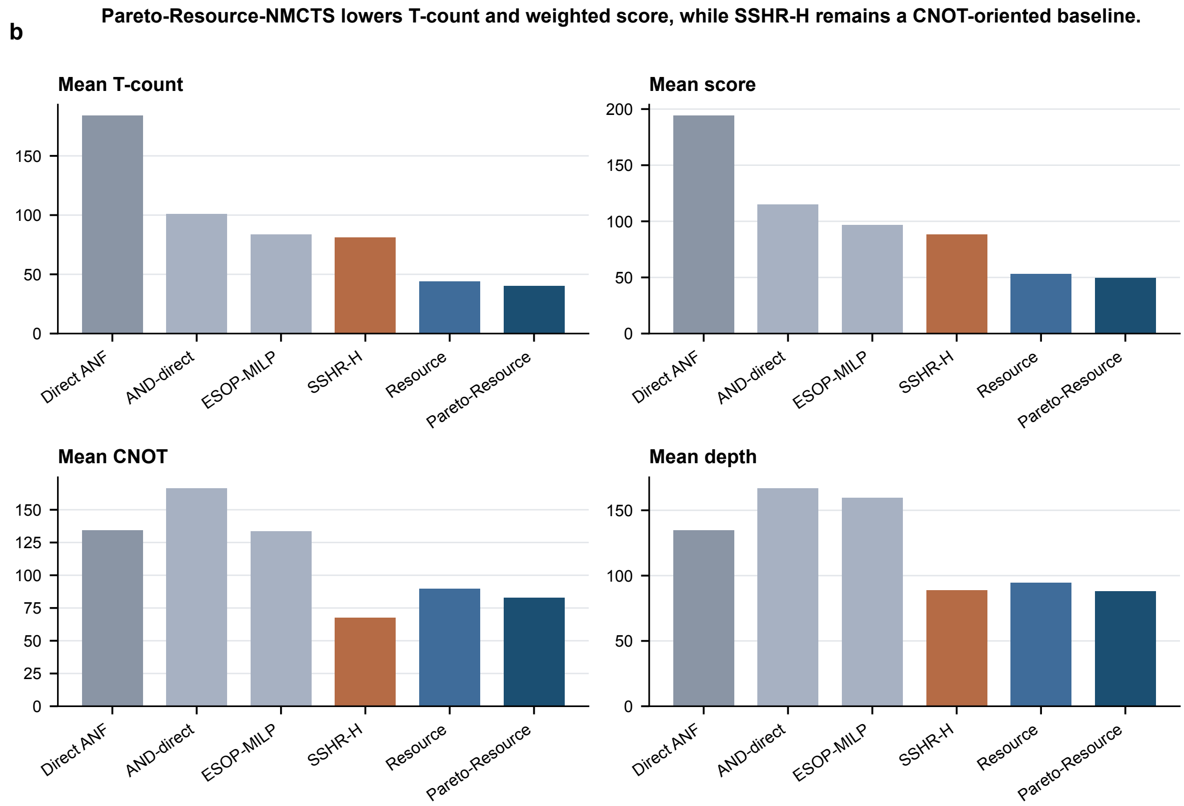

Figure 2: **Mean logical resources on the traditional slice.** Pareto-Resource-NMCTS has the lowest T-count and weighted score; SSHR-H remains a strong CNOT-oriented baseline.

Table 1: Mean logical resources on the $n \leq 6$ traditional slice.

Method	Mean T	Mean CNOT	Mean depth	Mean ancilla	Mean score
Direct ANF	184.1	134.2	134.6	1.33	194.3
AND-direct ANF	101.0	166.5	166.9	2.18	115.2
ESOP-MILP	83.6	133.5	159.6	2.32	96.7
SSHR-H	81.0	**67.6**	88.7	1.36	88.2
Resource-NMCTS	43.9	89.9	94.5	1.92	53.2
Pareto-Resource-NMCTS	**40.4**	83.0	**88.0**	2.03	**49.6**

6.2 External logic-network and reversible-synthesis probes

Figure 3 summarizes paired score changes against several external or traditional baselines. The ROS-style LUT proxy covers 309 functions and 927 K-sweep rows, all truth-table checked; Pareto-Resource-NMCTS wins 309/309 score comparisons against the best-K proxy. The mockturtle probe uses official headers to resynthesize KLUT networks into XAGs; Pareto-Resource-NMCTS has 166/11/0 score wins on the traditional set and 64/0/0 on the $n = 14$ set.

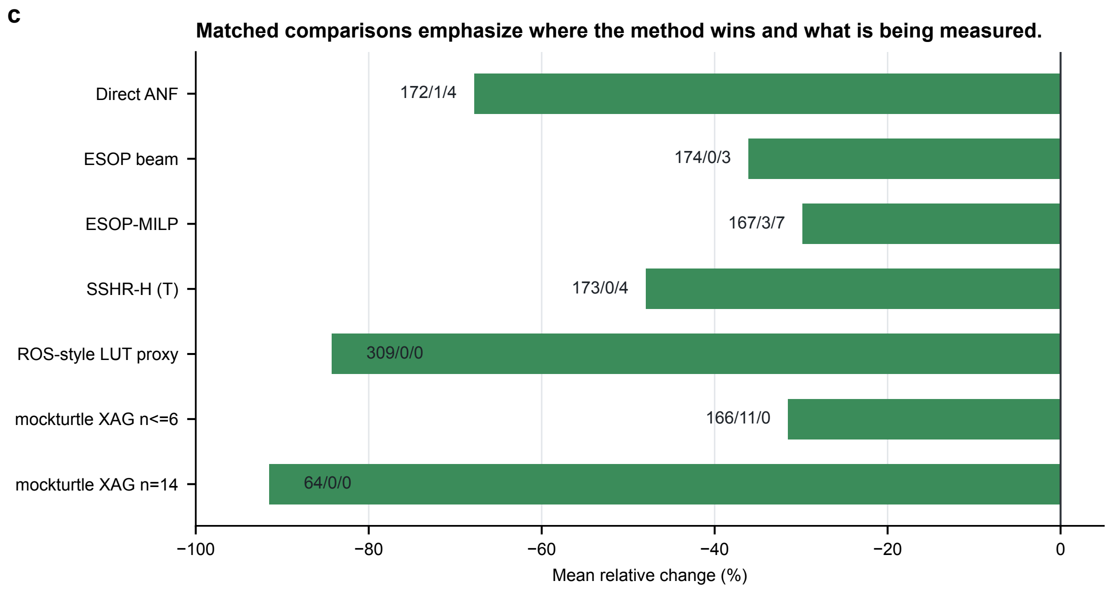

Figure 3: **Paired baseline comparisons.** Negative values favor the target method. Labels show win/loss/tie counts.

The CirKit probe converts the same BLIF benchmarks to AIGER with ABC, applies CirKit AIG rewriting and multiplicative-complexity analysis, writes Verilog, and verifies the Verilog readback through ABC. It produces 177/177 correct traditional rows and 64/64 correct $n = 14$ rows. Pareto-Resource-NMCTS wins all score comparisons but loses depth on most rows, making CirKit the strongest depth-oriented external probe in the current manuscript.

Table 2: CirKit 3 AIG/multiplicative-complexity probe on $n \leq 6$ functions.

Target vs CirKit AIG/MC	Items	W/L/T	Mean Δ
and_resource_nmcts	177	177/0/0	-60.84%
and_pareto_resource_nmcts	177	177/0/0	-62.34%
and_fprm_polarity_archive	177	177/0/0	-58.96%
and_direct_anf	177	124/53/0	-16.74%
direct_anf	177	46/131/0	+35.23%
and_esop_milp	177	150/27/0	-39.41%
sshr_h	177	164/13/0	-32.83%

Table 3: CirKit 3 AIG/multiplicative-complexity probe on the $n = 14$ set.

Target vs CirKit AIG/MC	Items	W/L/T	Mean Δ
and_resource_nmcts	64	64/0/0	-94.42%
and_profile_resource_nmcts	64	64/0/0	-94.42%
and_pareto_resource_nmcts	64	64/0/0	-94.46%
and_fprm_linear_pair	64	64/0/0	-94.27%
and_fprm_root_beam	64	64/0/0	-94.11%
and_direct_anf	64	64/0/0	-91.76%
direct_anf	64	64/0/0	-86.22%
and_fprm_greedy	64	64/0/0	-94.08%
and_affine_greedy	64	64/0/0	-94.08%

The RevKit CLI probe supplies a closer reversible-synthesis comparison. Each function is embedded as the exact oracle permutation $(x, y) \mapsto (x, y \oplus f(x))$, and legacy RevKit reads the SPEC permutation before running TBS, DBS, and RMS synthesis. All 531 direct flow rows return. The per-function best-score RevKit portfolio is strong on line count, but Pareto-Resource-NMCTS retains 173/0/4 score wins and a 67.28% mean score reduction.

Table 4: Legacy RevKit CLI reversible-synthesis portfolio on $n \leq 6$ functions. Each function uses the best-score result among TBS, DBS, and RMS.

Target vs RevKit CLI best-score	Items	W/L/T	Mean Δ
and_resource_nmcts	177	173/0/4	-66.32%
and_pareto_resource_nmcts	177	173/0/4	-67.28%
and_fprm_polarity_archive	177	173/0/4	-64.24%
and_direct_anf	177	167/6/4	-32.79%
direct_anf	177	87/86/4	+5.78%
and_esop_milp	177	172/1/4	-51.50%
sshr_h	177	170/7/0	+83.84%

6.3 Search contribution and ablation

Table 5 consolidates the contribution analysis. The largest pre-portfolio gain comes from affine and Boolean-ring structural search. The learned prior gives smaller but non-degrading quality gains on the traditional slice, while the frontier policies mainly control the quality/time frontier in high-dimensional term-set experiments.

Table 5: Matched contribution decomposition. Negative mean score changes favor the first method in each comparison.

Component	Dataset	Pairs	Score W/L/T	Mean Δ score
Affine greedy vs fixed-coordinate MCTS	ablation_affine	321	165/88/68	−12.12%
Neural refine over affine greedy	ablation_affine	322	65/0/257	−1.08%
Guarded Affine-NMCTS over no-guard	ablation_affine	322	88/0/234	−1.74%
Learned prior for Resource-NMCTS	traditional_resource	177	39/0/138	−1.10%
Pareto archive over Resource-NMCTS	traditional_resource	177	68/0/109	−3.26%
No-MCTS portfolio over heuristic-only	trad. ablation	177	54/0/123	−2.51%
Resource-NMCTS over no-MCTS portfolio	trad. ablation	177	54/0/123	−1.44%
Pareto Resource-NMCTS over no-MCTS portfolio	trad. ablation	177	106/0/71	−4.69%
Highdim no-MCTS portfolio over root beam	n14 ablation	16	14/0/2	−6.25%
Highdim no-MCTS portfolio over linear-pair	n14 ablation	16	14/0/2	−3.08%
Linear-pair guard vs root beam at n=14	highdim_resource	64	60/0/4	−3.00%
Recursive linear-pair guard vs root beam at n=15	n15 scale	32	30/0/2	−5.28%
Recursive linear-pair guard vs shallow linear-pair at n=16	n16 scale	24	23/0/1	−2.54%
Recursive linear-pair guard vs root beam at n=16	n16 scale	24	23/0/1	−4.31%
Baseline-preserving AI guard vs deterministic recursive guard at n=16	n16 scale	24	6/0/18	−0.05%
Fast linear-pair guard vs root beam at n=18	n18 stress	12	6/0/6	−1.91%
Resource-NMCTS vs fast linear-pair at n=18	n18 stress	12	12/0/0	−3.55%

6.4 Phase/Rz search

RevKit Python `oracle_synth` returns on all 177 traditional functions, but 171 rows contain non-Clifford R_z rotations, with 9242 such rotations in total. We therefore treat it as a phase/Rz lower-bound boundary rather than as an exact Clifford+T T-count comparison. The internal phase-parity emitter, fixed-polarity FPRM search, and affine FPRM phase search turn this boundary into a verifiable search branch.

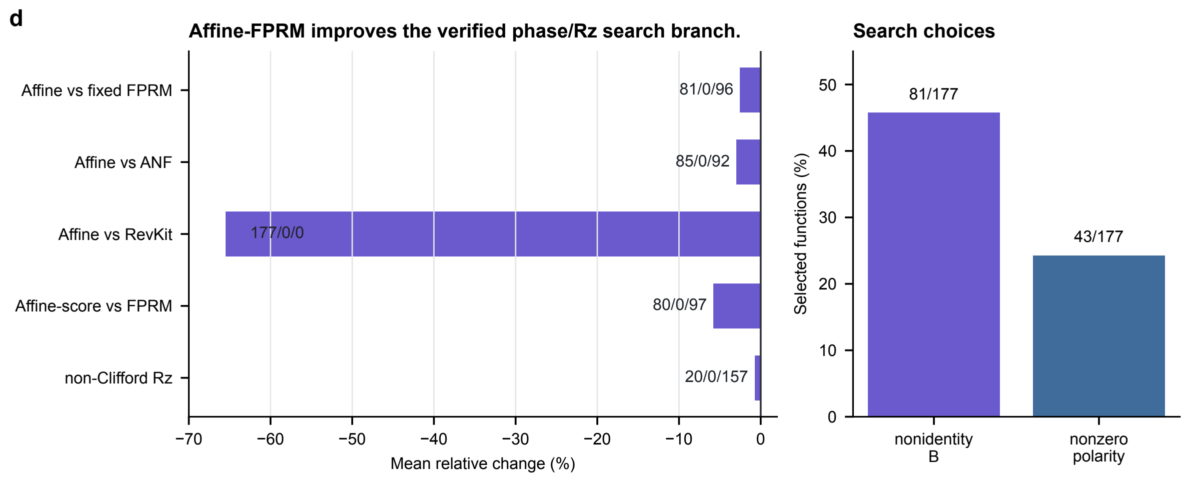

Figure 4: **Affine-FPRM phase search.** Affine preconditioning improves the phase-parity proxy over fixed-polarity FPRM and ANF baselines, while remaining a logical phase/Rz proxy rather than an approximate-rotation sequence.

Table 6: Affine-FPRM phase-parity search comparisons.

Comparison	Items	W/L/T	Mean Δ
Affine-FPRM-$T/R_z = 30$ vs FPRM	177	81/0/96	-2.51%
Affine-FPRM-$T/R_z = 30$ vs ANF	177	85/0/92	-2.98%
Affine-FPRM-$T/R_z = 30$ vs RevKit	177	177/0/0	-65.50%
Affine-FPRM non-Clifford Rz vs FPRM	177	20/0/157	-0.73%
Affine-FPRM-score vs FPRM	177	80/0/97	-5.77%

Increasing the affine transform budget from 32 to 128 yields additional but bounded gains: under the $T/R_z = 30$ proxy, the wide search gives 43/0/134 wins/losses/ties over budget 32 and reduces mean synthesis score by 0.60%. The learned phase candidate policy can recover nearly the wide-search quality with fewer exact evaluations, but same-budget random shortlists are close. Thus the result supports pruned phase-search feasibility, not a claim that the current neural scorer is already superior to random shortlisting.

Table 7: Affine-FPRM phase-search budget expansion.

Metric	Pairs	W/L/T	Mean Δ
score target	177	38/0/139	-1.59%
score+1/Rz target	177	40/0/137	-0.93%
$T/R_z = 30$ target	177	43/0/134	-0.60%
non-Clifford Rz	177	3/0/174	-0.29%
total Rz	177	35/2/140	-2.39%
CNOT	177	37/2/138	-1.74%
depth	177	41/2/134	-1.93%

Table 8: Learned phase candidate pruning on held-out $n = 6$ functions.

Comparison	Items	W/L/T	Mean Δ
Policy top-512 vs budget-32	38	17/0/21	-2.47%
Policy top-512 vs wide-128	38	0/6/32	+0.01%
Policy top-64 vs random top-64	38	12/4/22	+0.02%
Policy top-128 vs random top-128	38	10/5/23	+0.08%
Policy top-512 total Rz vs budget-32	38	16/0/22	-4.83%
Policy top-512 CNOT vs budget-32	38	16/1/21	-2.27%
Policy top-512 depth vs budget-32	38	16/1/21	-3.01%

6.5 High-dimensional verification

Figure 5 summarizes the current verification envelope. The manuscript-level evidence includes 5640/5640 item-level symbolic runs for $n = 20$–40, 1512/1512 width-probe symbolic runs, 531/531 phase selected rows, and 400/400 complete truth table bridge method rows. The complete bridge spans $n = 21$–25 and checks the oracle relation, the expanded ANF plan, and the emitted-circuit symbolic semantics.

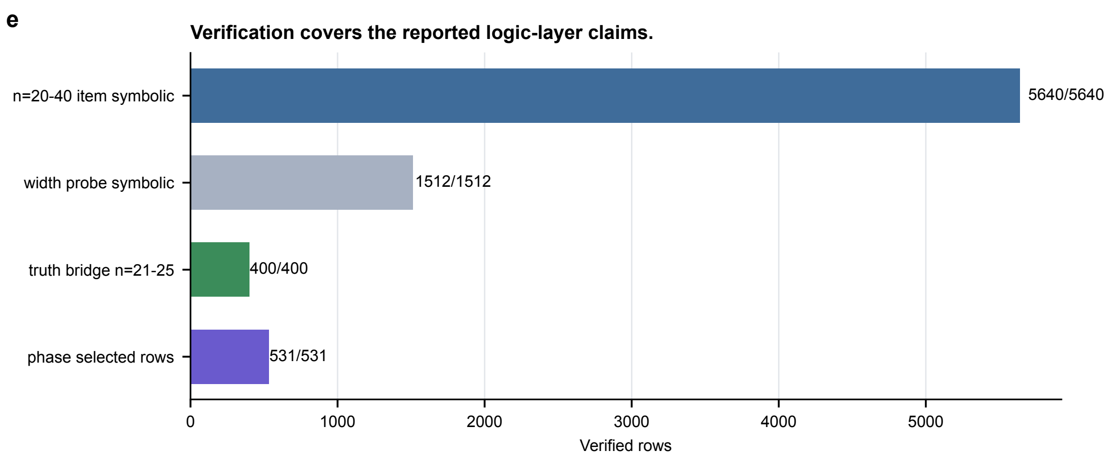

Figure 5: **Verification coverage.** Large term-set experiments are validated symbolically; $n = 21$–25 bridge slices add complete truth table checking on selected generated functions.

Table 9: Verification scope and boundary.

Check	Current coverage	Boundary
$n = 20$–40 item-level symbolic runs	5640/5640	Not full truth-table enumeration.
Width-probe symbolic runs	1512/1512	Supports the default action width, but remains term-set validation.
$n = 21$–25 complete bridge	400/400	Small generated slices; not exhaustive coverage of all high-dimensional functions.
Phase selected rows	531/531	Verified up to global phase; no rotation-sequence output.

7 Discussion

The evidence supports a specific, bounded claim: Resource-NMCTS is a logical-layer resource-constrained Boolean oracle synthesis method that improves T-count and weighted score over several fixed-representation and external-toolchain baselines. The claim is not that Resource-NMCTS dominates every metric. SSHR remains a strong CNOT-oriented small-function baseline; CirKit AIG/MC remains a strong depth-oriented external probe; and RevKit CLI uses fewer auxiliary lines on most traditional functions.

The ablations also calibrate the role of AI. The neural prior is useful, but the largest gain comes from the searchable algebraic action space and from guarded portfolio selection. This is a better fit for the evidence than claiming that deep reinforcement learning alone drives the result. In the phase/Rz branch, affine and polarity search give verified improvements, but same-budget random candidate shortlists remain close to the learned scorer. This leaves a clear next target: stronger phase/Rz policy objectives and rotation-spectrum optimization.

Several limitations are deliberate. All results are logical-layer estimates; the paper does not model routing, hardware connectivity, native gate sets, noise, or magic-state factories. The ROS-style LUT result is a proxy rather than a full ROS reproduction with SAT garbage management. The RevKit CLI baseline is a real exact-oracle reversible-synthesis probe, but CNOT/depth are derived from RevKit's Toffoli-control distribution and the experiment is still not a hardware mapping. High-dimensional symbolic checks validate emitted circuit semantics but do not replace full truth-table enumeration beyond the bridge slices.

8 Conclusion

We presented Resource-NMCTS, a neural Monte Carlo tree search framework for resource-constrained quantum Boolean oracle synthesis over verifiable ANF/FPRM actions. On matched small-function benchmarks, the method substantially lowers T-count and weighted score relative to direct ANF, ESOP, SSHR-H, and ESOP-MILP baselines. On external probes based on ROS-style LUT mapping, mockturtle, CirKit, and legacy RevKit CLI reversible synthesis, the method retains a strong weighted-score advantage while exposing clear depth and ancilla tradeoffs. Large symbolic experiments and $n = 21$–25 complete truth-table bridges support the semantic correctness of the logical-layer circuits. The next step is not to claim hardware-level optimality, but to strengthen the phase/Rz policy and complete fuller toolchain reproductions while keeping the claim at the logical synthesis layer.

References

[1] Robert K. Brayton and Alan Mishchenko. ABC: An academic industrial-strength verification tool. In *Computer Aided Verification*, volume 6174 of *Lecture Notes in Computer Science*, pages 24–40. Springer, 2010.

[2] David Kremer, Ali Javadi-Abhari, and Priyanka Mukhopadhyay. Optimizing the non-Clifford-count in unitary synthesis using reinforcement learning, 2025. arXiv:2509.21709.

[3] Mulundano Machiya, Matt Menickelly, Paul Hovland, and Ji Liu. MonteQ: A monte carlo tree search based quantum circuit synthesis framework, 2026. arXiv:2604.19029.

[4] Giulia Meuli, Mathias Soeken, Earl Campbell, Martin Roetteler, and Giovanni De Micheli. The role of multiplicative complexity in compiling low T-count oracle circuits. In *2019 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 1–8. IEEE, 2019.

[5] Giulia Meuli, Mathias Soeken, and Giovanni De Micheli. Xor-and-inverter graphs for quantum compilation. *npj Quantum Information*, 8(1):7, 2022.

[6] Giulia Meuli, Mathias Soeken, Martin Roetteler, and Giovanni De Micheli. ROS: Resource-constrained oracle synthesis for quantum computers. *Electronic Proceedings in Theoretical Computer Science*, 318:119–130, 2020.

[7] Sebastian Rietsch, Abhishek Y. Dubey, Christian Ufrecht, Maniraman Periyasamy, Axel Plinge, Christopher Mutschler, and Daniel D. Scherer. Unitary synthesis of Clifford+T circuits with reinforcement learning. In *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, pages 824–835. IEEE, 2024.

[8] Jordi Riu, Jan Nogué, Gerard Vilaplana, Artur Garcia-Saez, and Marta P. Estarellas. Reinforcement learning based quantum circuit optimization via ZX-calculus. *Quantum*, 9:1758, 2025.

[9] Mathias Soeken and collaborators. CirKit: Logic synthesis and optimization framework. https://github.com/msoeken/cirkit, 2026. CirKit 3 shell and legacy RevKit/CirKit command-line source.

[10] Mathias Soeken and collaborators. mockturtle: Logic network optimization and synthesis. https://mockturtle.readthedocs.io/, 2026. Official-header KLUT-to-XAG probe source.

[11] Mathias Soeken and collaborators. RevKit: Reversible logic and quantum circuit compilation. https://github.com/msoeken/revkit, 2026. Accessed as a local Python API and legacy command-line reversible-synthesis toolchain.

[12] Dimitrios Tsaras, Antoine Grosnit, Lei Chen, Zhiyao Xie, Haitham Bou-Ammar, and Mingxuan Yuan. ShortCircuit: AlphaZero-driven circuit design, 2024. arXiv:2408.09858.

[13] Robert Wille and Rolf Drechsler. BDD-based synthesis of reversible logic for large functions. In *Proceedings of the 46th Annual Design Automation Conference*, DAC '09, pages 270–275. ACM, 2009.

[14] Mingfei Yu, Alessandro Tempia Calvino, Mathias Soeken, and Giovanni De Micheli. Backend-aware fault-tolerant quantum oracle synthesis. In *2025 30th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 930–937. ACM, 2025.

[15] Qiang Zheng, Yongzhen Xu, Jiaxi Zhang, Zhaofeng Su, and Shenggen Zheng. CNOT oriented synthesis for small-scale boolean functions using spatial structures of parallelotopes. In *2025 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–9. IEEE, 2025.